

GUÍA PRÁCTICA - PRUEBAS UNITARIAS

React + Vite + Vitest + React Testing Library

Continuación proyecto mi-primer-react - DSY1104

Proyecto	Semana react / mi-primer-react
Objetivo	Incorporar pruebas automatizadas al proyecto React ya desarrollado.
Herramientas	Vitest, jsdom, React Testing Library, jest-dom y user-event.
Componentes a probar	Saludo.jsx, Producto.jsx y Login.jsx.
Metodología	Paso a paso: escribir, ejecutar, observar y explicar cada prueba.

Idea central: hasta ahora comprobábamos el funcionamiento mirando la aplicación y haciendo clic manualmente. En esta clase escribiremos código que compruebe automáticamente que los componentes continúan funcionando como esperamos.

Forma de trabajo: no avanzar todos los pasos de una vez. Después de cada prueba, guardar el archivo, mirar el resultado de Vitest y conversar sobre por qué pasó o falló.

Paso 1 - Abrir el proyecto actual

Abra en Visual Studio Code el mismo proyecto utilizado en las clases anteriores: **mi-primer-react**. No cree un proyecto nuevo.

Paso 2 - Comprobar que la aplicación funciona

```
npm run dev
```

Abra la dirección que muestra Vite, normalmente `http://localhost:5173`. Verifique que la aplicación actual funcione antes de agregar las pruebas.

Paso 3 - Detener el servidor de desarrollo

```
ctrl + c
```

Utilizaremos la terminal para instalar las dependencias de testing.

Paso 4 - Instalar Vitest

```
npm install -D vitest
```

Vitest será el framework que ejecutará nuestras pruebas. **-D** indica que se instala como dependencia de desarrollo.

Paso 5 - Instalar las herramientas para probar React

```
npm install -D @testing-library/react @testing-library/jest-dom @testing-library/user-event jsdom
```

Testing Library renderiza y consulta componentes; **jest-dom** agrega comprobaciones para el DOM; **user-event** simula acciones del usuario; **jsdom** simula el navegador.

Paso 6 - Configurar Vitest en vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],

  test: {
    environment: 'jsdom',
    setupFiles: ['./src/setupTests.js'],
    globals: true,
  },
})
```

Con **environment: 'jsdom'** nuestras pruebas disponen de un DOM simulado. **setupFiles** cargará una configuración común antes de las pruebas.

Paso 7 - Crear setupTests.js

Dentro de **src** cree el archivo **setupTests.js**:

```
import '@testing-library/jest-dom'
```

Esto permite utilizar expresiones como **toBeInTheDocument()** y **toHaveValue()**.

Paso 8 - Agregar el comando npm test

En **package.json**, dentro de scripts, agregue:

```
"test": "vitest"
```

Ejemplo:

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview",
  "test": "vitest"
}
```

Paso 9 - Crear la primera prueba para Saludo

Junto a **Saludo.jsx** cree **Saludo.test.jsx**:

```
import { render, screen } from '@testing-library/react'
import { describe, test, expect } from 'vitest'
import Saludo from './Saludo'

describe('Componente Saludo', () => {

  test('muestra el nombre recibido', () => {

    render(
      <Saludo nombre="Hernán" />
    )

    expect(
      screen.getByText('Hola Hernán')
    ).toBeInTheDocument()

  })

})
```

describe() agrupa pruebas; **test()** define un comportamiento; **render()** crea el componente; **screen** busca elementos; **expect()** expresa el resultado esperado.

Paso 10 - Ejecutar la primera prueba

```
npm test
```

Vitest debe detectar automáticamente **Saludo.test.jsx**. Si todo coincide, la prueba aparecerá como aprobada.

Paso 11 - Provocar un error intencional

Cambie temporalmente **Hola Hernán** por **Hola Pedro** en la expectativa. Observe que la prueba falla. Luego vuelva al texto correcto.

Este ejercicio demuestra que una prueba compara el **resultado real** con el **resultado esperado**.

Paso 12 - Definir qué vamos a probar en Producto

Antes de escribir código, identifique comportamientos: mostrar nombre y precio, mostrar stock inicial, aumentar stock, disminuir stock y evitar stock negativo.

Paso 13 - Crear Producto.test.jsx

Cree **src/components/Producto.test.jsx** junto a **Producto.jsx**.

Paso 14 - Preparar Producto dentro de un Router

Producto utiliza un **Link** de React Router. Para probarlo sin abrir el navegador utilizaremos **MemoryRouter**:

```
import { render, screen } from '@testing-library/react'
import { describe, test, expect } from 'vitest'
import { MemoryRouter } from 'react-router-dom'
import Producto from './Producto'

describe('Componente Producto', () => {

  test('muestra el nombre del producto', () => {

    render(
      <MemoryRouter>
        <Producto
          id="1"
          nombre="Notebook Lenovo"
          precio="599.990"
        />
      </MemoryRouter>
    )

    expect(
      screen.getByText('Notebook Lenovo')
    ).toBeInTheDocument()

  })

})
```

MemoryRouter proporciona el contexto de navegación que necesita **Link**, pero permanece en memoria y resulta apropiado para las pruebas.

Paso 15 - Probar el precio

```
test('muestra el precio del producto', () => {

  render(
    <MemoryRouter>
      <Producto
        id="1"
        nombre="Notebook Lenovo"
        precio="599.990"
      />
    </MemoryRouter>
  )

  expect(
    screen.getByText('Precio: $599.990')
  ).toBeInTheDocument()

})
```

Paso 16 - Evitar repetir el render del producto

Como estamos repitiendo el mismo bloque, cree antes de describe una función auxiliar:

```
const renderProducto = () => {

  render(
    <MemoryRouter>
      <Producto
        id="1"
        nombre="Notebook Lenovo"
        precio="599.990"
      />
    </MemoryRouter>
  )
}
```

Luego cada prueba puede comenzar simplemente con **renderProducto()**.

Paso 17 - Probar el stock inicial

```
test('muestra stock inicial de 5', () => {

  renderProducto()

  expect(
    screen.getByText('Stock disponible: 5')
  ).toBeInTheDocument()

})
```

Paso 18 - Limpiar localStorage antes de cada prueba

Producto utiliza localStorage. Para que una prueba no contamine a otra, agregue **beforeEach**:

```
import {
  describe,
  test,
  expect,
  beforeEach
} from 'vitest'

beforeEach(() => {
  localStorage.clear()
})
```

beforeEach() ejecuta una preparación antes de cada test. Cada prueba comienza desde un estado conocido.

Paso 19 - Importar userEvent

```
import userEvent from '@testing-library/user-event'
```

userEvent nos permitirá simular acciones reales del usuario, como clics y escritura.

Paso 20 - Probar que el botón + aumenta el stock

```
test('aumenta el stock al presionar +', async () => {

  const user = userEvent.setup()
```

```

renderProducto()

const botonAumentar = screen.getByRole(
  'button',
  { name: '+' }
)

await user.click(botonAumentar)

expect(
  screen.getByText('Stock disponible: 6')
).toBeInTheDocument()
})

```

Partimos en 5, simulamos un clic en + y esperamos encontrar **Stock disponible: 6**.

Paso 21 - Probar que el botón - disminuye el stock

```

test('disminuye el stock al presionar -', async () => {

  const user = userEvent.setup()

  renderProducto()

  const botonDisminuir = screen.getByRole(
    'button',
    { name: '-' }
  )

  await user.click(botonDisminuir)

  expect(
    screen.getByText('Stock disponible: 4')
  ).toBeInTheDocument()
})

```

Paso 22 - Probar la regla: el stock no puede ser negativo

```

test('no permite que el stock sea negativo', async () => {

  const user = userEvent.setup()

  renderProducto()

  const botonDisminuir = screen.getByRole(
    'button',
    { name: '-' }
  )

  await user.click(botonDisminuir)
  await user.click(botonDisminuir)
  await user.click(botonDisminuir)
  await user.click(botonDisminuir)
  await user.click(botonDisminuir)
  await user.click(botonDisminuir)

  expect(
    screen.getByText('Stock disponible: 0')
  ).toBeInTheDocument()
})

```

Esta prueba comprueba una **regla de negocio**, no solo la presencia de un texto.

Paso 23 - Revisar las pruebas de Producto

Al guardar, Vitest debería mostrar las pruebas de nombre, precio, stock inicial, aumento, disminución y límite en cero como aprobadas.

Paso 24 - Definir qué vamos a probar en Login

Probaremos que aparece el formulario, que se puede escribir en sus campos y que las validaciones reaccionan ante datos incorrectos y correctos.

Paso 25 - Crear Login.test.jsx

Cree `src/pages/Login.test.jsx`.

Paso 26 - Primera prueba de Login

```
import { render, screen } from '@testing-library/react'
import { describe, test, expect } from 'vitest'
import Login from './Login'

describe('Página Login', () => {

  test('muestra el formulario de inicio de sesión', () => {

    render(<Login />)

    expect(
      screen.getByText('Iniciar Sesión')
    ).toBeInTheDocument()

  })

})
```

Paso 27 - Asociar correctamente label e input

Para buscar los inputs por su etiqueta, el label debe estar asociado al input. En `Login.jsx` use:

```
<label
  htmlFor="email"
  className="form-label"
>
  Correo electrónico
</label>

<input
  id="email"
  type="text"
  className="form-control"
  value={email}
  onChange={(event) => setEmail(event.target.value)}
/>
```

Y para contraseña:

```
<label
  htmlFor="password"
  className="form-label"
>
  Contraseña
</label>

<input
  id="password"
  type="password"
  className="form-control"
  value={password}
  onChange={(event) => setPassword(event.target.value)}
/>
```

Además de facilitar las pruebas, esta asociación mejora la accesibilidad del formulario.

Paso 28 - Comprobar que existen los dos campos

```
test('muestra los campos de correo y contraseña', () => {

  render(<Login />)

  expect(
    screen.getByLabelText('Correo electrónico')
  ).toBeInTheDocument()

  expect(
    screen.getByLabelText('Contraseña')
  ).toBeInTheDocument()

})
```

Paso 29 - Importar `userEvent` en `Login.test.jsx`

```
import userEvent from '@testing-library/user-event'
```

Paso 30 - Probar escritura en el correo

```
test('permite escribir el correo electrónico', async () => {

  const user = userEvent.setup()

  render(<Login />)

  const inputEmail =
    screen.getByLabelText('Correo electrónico')

  await user.type(
    inputEmail,
    'hernan@gmail.com'
  )

  expect(inputEmail).toHaveValue(
    'hernan@gmail.com'
  )

})
```

user.type() simula la escritura del usuario. El onChange de React actualiza el estado y luego comprobamos el valor del input.

Paso 31 - Probar escritura en la contraseña

```
test('permite escribir la contraseña', async () => {

  const user = userEvent.setup()

  render(<Login />)

  const inputPassword =
    screen.getByLabelText('Contraseña')

  await user.type(
    inputPassword,
    '12345'
  )

  expect(inputPassword).toHaveValue('12345')

})
```

Paso 32 - Probar un correo inválido

```
test('muestra error cuando el correo no es válido', async () => {

  const user = userEvent.setup()

  render(<Login />)

  await user.type(
    screen.getByLabelText('Correo electrónico'),
    'hernan'
  )

  await user.type(
    screen.getByLabelText('Contraseña'),
    '12345'
  )

  await user.click(
    screen.getByRole(
      'button',
      { name: 'Ingresar' }
    )
  )

  expect(
    screen.getByText(
      'El correo electrónico no es válido'
    )
  ).toBeInTheDocument()

})
```

Paso 33 - Probar una contraseña demasiado corta

```
test('muestra error cuando la contraseña es muy corta', async () => {

  const user = userEvent.setup()
```

```

render(<Login />)

await user.type(
  screen.getByLabelText('Correo electrónico'),
  'hernan@gmail.com'
)

await user.type(
  screen.getByLabelText('Contraseña'),
  '12'
)

await user.click(
  screen.getByRole(
    'button',
    { name: 'Ingresar' }
  )
)

expect(
  screen.getByText(
    'La contraseña debe tener al menos 4 caracteres'
  )
).toBeInTheDocument()
})

```

Paso 34 - Probar datos válidos

```

test('no muestra error con datos válidos', async () => {

  const user = userEvent.setup()

  render(<Login />)

  await user.type(
    screen.getByLabelText('Correo electrónico'),
    'hernan@gmail.com'
  )

  await user.type(
    screen.getByLabelText('Contraseña'),
    '12345'
  )

  await user.click(
    screen.getByRole(
      'button',
      { name: 'Ingresar' }
    )
  )

  expect(
    screen.queryByText(
      'El correo electrónico no es válido'
    )
  ).not.toBeInTheDocument()

  expect(
    screen.queryByText(
      'La contraseña debe tener al menos 4 caracteres'
    )
  ).not.toBeInTheDocument()
})

```

getBy... se usa cuando esperamos encontrar un elemento. **queryBy...** es útil cuando queremos comprobar que un elemento no está presente.

Paso 35 - Ejecutar todas las pruebas

```
npm test
```

Vitest debe ejecutar los archivos **Saludo.test.jsx**, **Producto.test.jsx** y **Login.test.jsx**. Revise cada resultado antes de continuar.

Paso 36 - Interpretar PASS y FAIL

Una prueba en verde indica que el comportamiento observado coincide con el esperado. Una prueba en rojo no significa automáticamente que Vitest esté mal: puede haber un error en el componente, en la prueba o una diferencia entre lo que esperábamos y lo que realmente hace la aplicación.

Paso 37 - Relacionar las herramientas utilizadas

Herramienta	¿Para qué la usamos?
describe()	Agrupar pruebas relacionadas.
test()	Definir un comportamiento que queremos comprobar.
render()	Renderizar un componente React dentro de la prueba.
screen	Buscar elementos visibles o accesibles.
expect()	Indicar el resultado esperado.
beforeEach()	Preparar el entorno antes de cada prueba.
userEvent	Simular clics y escritura del usuario.
MemoryRouter	Proporcionar Router a componentes que usan Link.
jsdom	Simular un DOM/navegador para las pruebas.

Paso 38 - Cierre de la actividad

Al finalizar, los alumnos deben poder explicar la diferencia entre **ejecutar manualmente una aplicación y comprobar automáticamente sus comportamientos mediante pruebas**.

Idea de cierre: las pruebas permiten detectar regresiones. Si más adelante modificamos Producto o Login, podemos ejecutar **npm test** para comprobar si los comportamientos que antes funcionaban continúan funcionando.

Estructura esperada del proyecto

```
src/
├── components/
│   ├── Contador.jsx
│   ├── Navbar.jsx
│   ├── Producto.jsx
│   ├── Producto.test.jsx
│   ├── Saludo.jsx
│   └── Saludo.test.jsx
├── pages/
│   ├── DetalleProducto.jsx
│   ├── Inicio.jsx
│   ├── Login.jsx
│   ├── Login.test.jsx
│   └── Productos.jsx
├── App.jsx
├── main.jsx
└── setupTests.js
```

Secuencia conceptual de la clase

1. **Saludo:** primera prueba simple - renderizar y buscar texto.
2. **Producto:** props, estado, interacción con botones y regla de negocio.
3. **Login:** formulario controlado, escritura del usuario y validaciones.
4. **Vitest:** ejecutar todas las pruebas y detectar posibles regresiones.

Comandos utilizados

```
npm run dev

npm install -D vitest

npm install -D @testing-library/react @testing-library/jest-dom @testing-library/user-event jsdom

npm test
```

Próxima extensión sugerida: probar explícitamente la persistencia de Producto con localStorage y posteriormente probar DetalleProducto/useParams. Estos temas pueden incorporarse como continuación una vez que los alumnos dominen las pruebas básicas de esta guía.